Answer Engineering: Local Trajectory Editing for Protocol-Constrained Decision Making in Large Language Models

Victor Lavrenko
PeaceTech VC, Israel
victor@peacetech.vc

Anastasiia Molodnitskaia, MD
Department of Otolaryngology–Head and Neck Surgery
Rambam Health Care Campus, Haifa, Israel
a_molodnitskaia@rmc.gov.il

April 30, 2026

Abstract

Large language models can produce confident but wrong answers when they violate procedures in domains where compliance is critical for decisions. Step-by-step reasoning improves structure and transparency but often shifts the distribution of errors rather than preventing them because procedures remain systematically underused. This effect arises across multiple protocol-driven domains; here, we evaluate it on a clinical benchmark requiring differential diagnosis as a proof-of-concept, using a binary adherence endpoint. In this task, compliant outcomes for the target condition decreased from 54.5% under unguided generation to 25.1% when a prompt-engineering baseline enforcing step-by-step reasoning was applied, while performance on the contrastive condition increased from 1.6% to 58.9%. We hypothesize that reliability can be improved by intervening locally during generation—both selecting valid reasoning trajectories and enforcing required tokens when violations occur. This hypothesis is implemented as a runtime control mechanism in which rules enforce procedural compliance without retraining models or performing global search. In this workflow, localized trajectory intervention increased compliant outcomes for the target condition from 25.1% to 83.5% and improved performance on the contrastive condition from 58.9% to 77.9%, suggesting that failures are consistent with localized structural violations that can be corrected through domain-specific protocols.

1 Introduction

Modern large language models (LLMs) generate fluent explanations and decisions, and explicit reasoning has emerged as a widely used strategy for improving model accuracy and reliability. Techniques such as chain-of-thought prompting, structured reasoning, and agent-based planning have demonstrated measurable performance gains across a range of tasks, particularly those requiring multi-step inference or procedural decision-making (Wei et al., 2022; Yao, J. Zhao, et al., 2023; Yao, Yu, et al., 2023; Shinn et al., 2023). As a result, generating reasoning has become a common mechanism for improving both model capability and interpretability.

However, subsequent research has shown that the presence of reasoning does not guarantee that the reasoning is faithful to the decision process that produces the final answer. Empirical studies have documented that language models frequently generate explanations that function as rationalizations rather than causal justifications of their outputs (Lyu et al., 2023; Paul et al., 2024). In such cases, reasoning steps may be internally inconsistent, incompatible with domain rules, or disconnected from the mechanism that determines the final prediction. Even models explicitly trained to produce structured reasoning remain only partially faithful to the underlying decision process (Yun et al., 2025). These findings suggest that reasoning alone is not a sufficient reliability mechanism. In practice, structured reasoning may shift the distribution of errors rather than eliminate them, improving performance for some conditions while introducing new procedural failures for others.

A practical manifestation of this limitation arises from the temporal structure of generated reasoning. Models may commit to a decision early and then generate reasoning steps that rationalize the chosen answer, producing a logically inconsistent explanation of the decision process. Conversely, when reasoning is generated before the decision, intermediate steps may be irrelevant, incomplete, or disconnected from the mechanism that determines the final outcome. In both cases, reasoning is present but does not reliably guide

the decision process or ensure protocol-compliant outcomes.

This limitation is particularly consequential in domains governed by explicit procedural requirements. In regulated settings, correctness depends not only on factual accuracy but also on adherence to mandated reasoning steps defined by professional protocols. An answer may therefore appear plausible yet still be operationally invalid if required checks or distinctions are omitted. Despite their linguistic competence, current LLMs do not reliably enforce such protocol constraints during generation (Wei et al., 2022; Yao, J. Zhao, et al., 2023; Y. Liu et al., 2023; Lyu et al., 2023).

These observations suggest that improving reliability in protocol-driven domains requires mechanisms that operate during generation rather than only after an answer has been produced.

Evidence from clinical decision-making—one of the most extensively studied protocol-governed domains—illustrates the magnitude of this challenge. In a benchmark evaluating guideline adherence across 17 large language models and 20 diagnostic scenarios spanning 13 medical specialties, the best-performing model achieved only 41.9/50 on a standardized protocol-adherence score (Fast et al., 2024). In a separate study evaluating treatment-selection decisions from real-world medical reports, agreement between model recommendations and an expert heart team ranged from $\kappa = -0.47$ to 0.22, indicating poor concordance on realistic inputs (Roeschl et al., 2025). Additional studies report that LLMs frequently fail to follow diagnostic or treatment guidelines in clinical decision tasks and may generate confident recommendations that diverge from established protocols (Hager et al., 2024). Together these findings demonstrate that fluent reasoning does not guarantee protocol-compliant outcomes.

Because LLM outputs are generated sequentially, reasoning unfolds as a trajectory of intermediate statements that can serve as a runtime control surface for enforcing procedural correctness. Let

$$\mathcal{R} = \{r_1, \ldots, r_n\}$$

denote the reasoning trajectory generated by the model, where the tokens realizing each step are sampled from the same autoregressive distribution P_θ conditioned on the problem statement and the previously generated steps.

We use $\mathcal{R}$ for the human-readable sequence of reasoning steps and $T = x_{1:t}$ for the token-level prefix operated on by the runtime.

In practice, reasoning trajectories may contain localized failures—for example, when a required diagnostic distinction is omitted or a mandatory procedural step is skipped. Such errors may affect both intermediate reasoning and the final answer delivered to the user. Empirical evidence suggests that improving reasoning faithfulness, verification, or structured evaluation often improves downstream performance (Lyu et al., 2023; Paul et al., 2024; Yun et al., 2025). These observations suggest that some observed failures may reflect structural violations of procedural requirements rather than insufficient domain knowledge alone.

This observation motivates the need for mechanisms capable of intervening directly within the reasoning trajectory during generation.

Such mechanisms must be configurable by domain experts and interpretable in operational settings, where procedural requirements evolve and must remain auditable. Accordingly, trajectory intervention is implemented through a small set of explicit rule families that define permissible edits to the reasoning trajectory during generation.

This work addresses these failures by applying localized runtime edits to the reasoning trajectory during generation rather than after generation has been completed.

To formalize this idea, we distinguish between the generated reasoning trajectory and the controlled reasoning trajectory. Let

$$\mathcal{R}^* = \{r_1^*, \ldots, r_n^*\}$$

denote the trajectory after runtime interventions. A controlled reasoning step r_i^* represents the accepted version of step i after possible editing, replacement, or regeneration to satisfy domain-specific constraints. The final answer is produced from the controlled trajectory:

$$y \sim P_\theta(\cdot \mid s, \mathcal{R}^*)$$

Local edits redefine the visible prefix used by the autoregressive model, allowing generation to continue from a corrected state without requiring full regeneration of the reasoning sequence. This mechanism operationalizes the hypothesis that reliability can be improved by correcting localized protocol violations during generation without retraining models or performing global search.

A growing line of work has explored methods for improving reasoning and control in LLMs, including prompting strategies and agent-based reasoning frameworks (Wei et al., 2022; Yao, J. Zhao, et al., 2023; Yao, Yu, et al., 2023; Shinn et al., 2023), principle-based alignment and self-critique (Bai et al., 2022), and decoding-time control methods that guide generation behavior without retraining, such as constrained or

guided decoding and inference-time alignment mechanisms (Yang and Klein, 2021; Shi et al., 2023; Srivatsa et al., 2024). These approaches primarily influence generation globally or probabilistically. In contrast, the method proposed in this paper intervenes locally during generation by enforcing step-level protocol rules within the reasoning trajectory itself.

A critical open question is whether enforcing procedural constraints during generation can measurably improve protocol-compliant outcomes compared with reasoning alone.

To evaluate this question, we apply the approach to a controlled clinical reasoning benchmark involving sudden sensorineural hearing loss (SSNHL), where protocol violations are clinically consequential and operationally measurable. The benchmark uses template-generated paired cases evaluated under fixed decoding conditions and compares unguided generation, reasoning-prompt generation, and trajectory-controlled generation to measure the effect of runtime protocol enforcement under matched conditions.

Although the empirical evaluation in this paper focuses on a clinical protocol, the trajectory-control mechanism itself is not domain-specific. The framework is implemented as an executable runtime system with observable telemetry and rule-based configuration. For example, the repository quickstart notebook includes a software-engineering prompt about rate limiting in a small Python web API service, where a local replacement rule redirects a from-scratch implementation plan toward reuse of an existing library.[1]

This work therefore investigates whether localized trajectory intervention can provide the runtime control required for reliable protocol-compliant reasoning.

To address these requirements, this paper makes three contributions:

1. A decoding-time runtime mechanism for enforcing protocol compliance through localized trajectory intervention during generation, without retraining models or performing global search.

2. A rule-based control abstraction for specifying domain-specific procedural constraints as interpretable trajectory-editing operations.

3. An empirical evaluation demonstrating that step-by-step reasoning alone may reduce protocol compliance in structured decision tasks and that localized trajectory intervention substantially improves protocol-compliant outcomes.

[1] See the repository quickstart notebook `notebooks/quickstart.ipynb`.

2 Failure Modes in Protocol-Constrained Generation

Protocol-constrained reasoning requires not only factual correctness but also explicit adherence to externally defined procedural steps. Empirical studies suggest that language models often possess relevant domain knowledge yet fail to consistently apply required procedural checks during generation. Analyses of reasoning faithfulness and guideline adherence indicate that such violations frequently arise within the visible reasoning trajectory itself—for example when intermediate steps omit required distinctions or skip mandatory verification procedures (Lanham et al., 2023; Arcuschin et al., 2025; Fast et al., 2024). Despite strong benchmark performance, LLMs therefore often produce fluent answers that violate protocol requirements. We summarize five recurring failure modes (F1–F5) documented in the literature.

F1: Unfaithful Visible Reasoning

Reasoning traces produced by language models are often not faithful to the decision process that generates the final answer. Studies of chain-of-thought reasoning show that models can produce plausible intermediate explanations that do not reflect the actual factors driving the prediction (Lanham et al., 2023; Lyu et al., 2023). Subsequent work shows that explanations generated in the wild frequently diverge from the model's internal computation and may function primarily as post-hoc rationalizations (Arcuschin et al., 2025).

F2: Prompt and Stem Drift

During multi-step generation, the reasoning trajectory may gradually diverge from the original task specification. As generation progresses, the model may omit critical constraints, forget key details from the case description, or reinterpret variables in ways that detach the reasoning from the original prompt. Such drift can arise in multi-step reasoning frameworks, including tree-based and agent-based approaches, because long reasoning trajectories may gradually detach from the original task specification (Yao, Yu, et al., 2023; Yao, J. Zhao, et al., 2023).

F3: Rationalization of Chosen Answers

Language models may generate explanations that justify an answer rather than derive it. In such cases, the explanation is constructed to rationalize a prediction that has already been selected, rather

than reflecting the reasoning process that produced it. This behavior is related to unfaithful reasoning traces but differs in that the explanation serves primarily as post-hoc justification. In protocol-constrained settings, this can be especially problematic because the resulting explanation may appear systematic while failing to actually perform the required procedural analysis (Lanham et al., 2023; Arcuschin et al., 2025).

F4: Local Protocol Inconsistencies

A recurring failure pattern in LLM reasoning is that an explanation may remain globally plausible while containing a local error at a protocol-critical step. Even when the reasoning trajectory broadly follows the intended analysis, a single incorrect inference or protocol misinterpretation can invalidate the resulting conclusion. In protocol-constrained tasks, such errors may involve misstating a key distinction, using terminology that does not match the triggering condition, or introducing a locally inconsistent inference (Turpin et al., 2023; Lyu et al., 2023; Paul et al., 2024; Wiegreffe et al., 2024).

F5: Omission of Mandatory Protocol Steps

Language models can also omit procedural steps required by domain protocols. Even when a model correctly interprets much of the problem, its generated reasoning may fail to explicitly perform a required verification step, diagnostic distinction, or safety check. This pattern is consistent with the broader observation that many current reasoning methods improve answer quality without explicitly guaranteeing satisfaction of externally specified procedural constraints (Yao, J. Zhao, et al., 2023; Yao, Yu, et al., 2023; Srivatsa et al., 2024; Bai et al., 2022).

Taken together, these failure modes suggest that protocol violations frequently arise within the visible reasoning trajectory itself. Models may drift from the case description, rationalize a chosen conclusion, introduce a local inconsistency, or omit a required procedural step. This motivates mechanisms that intervene during generation rather than relying solely on prompting or post-hoc validation.

3 Existing Approaches and Challenges for Protocol-Constrained Generation

A large body of work has improved the reliability of large language models (LLMs) through prompting strategies, structured reasoning frameworks, sampling methods, constrained decoding, and symbolic control (Wei et al., 2022; Yao, J. Zhao, et al., 2023; Wang et al., 2023; J. Chen et al., 2025; Y. Liu et al., 2023; H. Zhao et al., 2024). These approaches achieve substantial gains on reasoning benchmarks and remain strong baselines for many applications.

However, when evaluated in protocol-governed settings, these methods address protocol compliance only indirectly. Protocol-constrained reasoning requires not only producing a correct final answer but also performing specific procedural checks or reasoning steps required by external rules, guidelines, or regulations.

When analyzed through the lens of the failure modes introduced in Section 2—unfaithful reasoning (F1), prompt drift (F2), rationalization (F3), local protocol inconsistencies (F4), and omission of mandatory protocol steps (F5)—existing approaches exhibit uneven coverage. Table 1 summarizes how representative classes of methods mitigate these failure modes.

Beyond empirical coverage of failure modes, existing approaches can also be understood according to the *mechanism by which they influence the reasoning trajectory*. Broadly, three intervention paradigms appear in the literature:

- **Generation biasing**, which influences reasoning indirectly through prompts, instructions, or structural scaffolds that encourage certain reasoning patterns.
- **Trajectory selection**, which generates multiple candidate reasoning paths and selects or reranks them according to sampling criteria or learned reward models.
- **Explicit constraint enforcement**, which imposes structural or symbolic rules on the reasoning process itself.

These paradigms correspond to different control points in the generation pipeline and therefore differ in their ability to guarantee adherence to domain protocols.

While Table 1 summarizes empirical coverage of protocol-related failure modes, these approaches intervene at different stages of the reasoning process. Some bias generation through prompts or sampling strategies, others evaluate or rerank candidate trajectories after they are produced, and symbolic systems replace learned reasoning with explicit decision rules.

Table 2 therefore reorganizes several of the above method families according to their *mechanistic intervention point*. Rather than reproducing the full taxonomy of Table 1, the table highlights representative intervention styles that differ most clearly in how they influence the generation process.

Method class	**F1**	**F2**	**F3**	**F4**	**F5**
Prompting / CoT Wei et al., 2022	○	○	○	–	–
Self-consistency / sampling Wang et al., 2023	○	○	○	–	–
ReAct / agent reasoning Yao, J. Zhao, et al., 2023; Shinn et al., 2023	○	○	○	–	–
Structured reasoning / scaffolds J. Chen et al., 2025; Yao, Yu, et al., 2023	○	○	○	–	–
Verification / correction / calibration Deng et al., 2023; Tyen et al., 2024	○	–	○	–	–
Principle-based alignment / self-critique Bai et al., 2022	○	○	○	–	–
Process reward models / trajectory scoring Lightman et al., 2023; Yun et al., 2025	○	○	○	○	–
Structured-output and guided decoding Yang and Klein, 2021; Shi et al., 2023; Koo et al., 2024; H. Zhao et al., 2024; Srivatsa et al., 2024	–	–	–	○	○
Rule-based / expert systems Y. Liu et al., 2023; H. Zhao et al., 2024	–	–	–	✓	✓
Answer Engineering (this work)	○	✓	○	✓	○

Table 1: Coverage of protocol failure modes by representative classes of approaches. F1: unfaithful reasoning; F2: prompt drift; F3: rationalization; F4: local protocol inconsistencies; F5: omission of mandatory protocol steps. ✓ strong coverage, ○ partial or indirect mitigation, – no direct coverage. For constrained decoding, partial coverage applies only when mandatory protocol elements can be encoded explicitly in the constrained output representation.

Table 2: Trajectory-time reasoning operations in LLM-based generation methods.

Method	Primary mechanism	Ops.†					Main limitation
		1	**2**	**3**	**4**	**5**	
Prompting / Chain-of-Thought	Prompt instructions and demonstrations	✓	✗	✗	✗	✗	Encourages structured reasoning but cannot enforce or repair incorrect steps Wei et al., 2022; Wang et al., 2023
Reflection / retry	Iterative self-critique after answer generation	○	○	○	○	○	Repairs occur after an answer exists rather than during generation Shinn et al., 2023; Bai et al., 2022
Verification / auditing	External validators or checkers	✗	○	✗	○	✗	Detects violations but typically does not alter reasoning in real time Lanham et al., 2023; Lyu et al., 2023
Constrained decoding	Token-level constraints or classifiers	○	○	○	✗	✓	Controls token admissibility rather than reasoning validity Yang and Klein, 2021; Koo et al., 2024; Srivatsa et al., 2024
Trajectory search	Selection among sampled reasoning paths	○	○	✗	✗	○	Chooses among alternatives but does not locally repair a trajectory Yao, Yu, et al., 2023; Wang et al., 2023
Answer Engineering	**Local protocol constraints applied during decoding**	✓	✓	✓	✓	✓	Requires authored protocol rules but preserves probabilistic model flexibility

Trajectory-time operations†	
1	Require explicit step-by-step reasoning during generation.
2	Avoid reasoning steps that violate protocol or domain constraints.
3	Modify or replace the reasoning step currently being generated.
4	Retrospectively modify previously generated reasoning before generation continues.
5	Resume generation from the corrected reasoning trajectory.

✓ = direct support; ○ = partial support; ✗ = not supported.

†Trajectory-time operations act directly on the evolving reasoning trajectory during generation, rather than only on prompts, outputs, or candidate selection.

3.1 Relation to Contemporary Runtime Control Frameworks

Recent inference and orchestration frameworks provide increasingly rich mechanisms for controlling language-model behavior at runtime. These systems expose structured decoding constraints, workflow policies, and programmable guardrails that significantly improve reliability in production deployments. Answer Engineering operates within this same design space, but targets a different control surface: local-

ized semantic repair of reasoning trajectories during generation.

Structured-output and guided-decoding systems. Modern inference engines such as vLLM and SGLang support structured-output generation using grammars, JSON schemas, regular expressions, and other admissibility constraints (Kwon et al., 2023; Zheng et al., 2024). These mechanisms ensure that generated text satisfies formal structural requirements and are widely used in production systems for enforcing output validity. Libraries such as Guidance, LMQL, and related constrained-decoding toolkits provide programmable interfaces for structured generation that allow developers to define grammar-based or schema-based constraints on the generation process (Research, 2023; Beurer-Kellner et al., 2023).

These approaches are highly effective when correctness can be expressed as admissibility of token sequences or output structure. However, structural constraints do not directly address semantic validity of intermediate reasoning steps. In protocol-constrained settings, many failures arise from locally inconsistent inferences rather than invalid syntax or formatting. Answer Engineering therefore operates at a different level of control: it modifies the evolving reasoning trajectory itself when protocol violations are detected, rather than restricting the set of admissible tokens.

Programmable guardrail and orchestration systems. Frameworks such as NeMo Guardrails provide configurable policies governing model inputs, outputs, and tool interactions (NVIDIA, 2023). These systems enable developers to define workflow-level rules, safety checks, and intervention logic that can redirect or block model behavior based on application policies. Such guardrail mechanisms are widely used to improve system reliability and enforce domain-specific constraints in deployed applications.

Answer Engineering differs in that it performs direct trajectory modification during decoding rather than enforcing policies only at workflow boundaries. Instead of filtering or rejecting completed outputs, the runtime intervenes locally within the reasoning sequence and resumes generation from the corrected prefix. This intervention model targets protocol violations at the point where they occur, enabling localized repair without discarding the remainder of the trajectory.

Positioning within the runtime control landscape. These approaches are complementary rather than mutually exclusive. Structured-output and guided-decoding systems ensure formal validity of generated text, guardrail frameworks enforce workflow-level policies, and trajectory editing provides localized semantic correction of reasoning steps during generation. In practical deployments, trajectory-level intervention can be layered on top of structured decoding or guardrail mechanisms as an additional reliability layer.

Prompt-based methods such as chain-of-thought prompting influence generation by encouraging models to produce intermediate explanations that structure the reasoning process (Wei et al., 2022). Sampling strategies such as self-consistency further improve robustness by selecting answers that remain stable across multiple reasoning trajectories (Wang et al., 2023). Agent-style reasoning frameworks such as ReAct combine reasoning with tool use and external actions, often improving task-solving performance (Yao, J. Zhao, et al., 2023). Structured reasoning frameworks attempt to organize reasoning into more explicit intermediate structures (J. Chen et al., 2025). While these approaches can reduce some forms of reasoning instability, they do not by themselves guarantee protocol compliance during generation.

Process reward models and trajectory-scoring methods evaluate intermediate reasoning steps using learned reward functions and external evidence, preferring trajectories that receive higher scores (Lightman et al., 2023; Yun et al., 2025). These methods can improve overall reasoning quality and alignment with domain knowledge, but they typically operate by selecting among candidate trajectories rather than directly repairing violations within the evolving reasoning process.

Structured-output and guided decoding methods enforce structural constraints on model outputs, for example ensuring that generated text satisfies a grammar or schema (Koo et al., 2024). These approaches are especially effective when correctness depends on formal structure, such as grammars or schemas. Their applicability to context-sensitive protocol constraints depends on whether those constraints can be encoded explicitly enough at decoding time.

Expert systems and rule-based controllers enforce domain protocols through explicit symbolic rule structures (Y. Liu et al., 2023; H. Zhao et al., 2024). These systems can guarantee the presence of mandatory protocol steps, but they can require substantial symbolic specification and maintenance effort as protocols evolve.

These comparisons indicate that most existing approaches influence reasoning through probabilistic guidance, structural constraints, or post-hoc validation rather than direct semantic modification of the reasoning trajectory itself.

These limitations motivate the approach summarized in Tables 1 and 2. In this work we develop *Answer Engineering*, a decoding-time framework that enforces protocol constraints through local intervention in the evolving reasoning trajectory using declarative domain rules. The trajectory-control mechanism is described in the following sections.

4 Decoding-Time Control of Reasoning Trajectories

Section 1 introduced trajectory control as the general decoding-time principle and trajectory editing as its concrete runtime mechanism. This section formalizes the approach and explains how prefix-conditioned decoding enables localized interventions during generation.

4.1 Prefix-conditioned decoding

Answer Engineering relies on a basic property of autoregressive decoding: future-token distributions depend on the visible prefix. Therefore, local edits to the generated trajectory redefine the context for subsequent generation without retraining the model. This makes runtime trajectory correction feasible for protocol-constrained reasoning.[2]

Observation: Prefix-conditioned continuation. In an autoregressive transformer, the probability of future tokens depends only on the visible prefix. Let $P_\theta(x_{t+1} \mid x_{1:t})$ denote the next-token distribution of an autoregressive language model. Then the distribution of all future tokens $x_{t+1:T}$ depends only on the visible prefix $x_{1:t}$. In particular, if a subsequence $x_{i:j}$ in the prefix is replaced by an edited segment $\tilde{x}_{i:j}$, the future generation distribution becomes

$$P_\theta(x_{t+1:T} \mid x_{1:i-1}, \tilde{x}_{i:j}, x_{j+1:t}),$$

which is identical to the distribution obtained if the edited prefix had been generated directly by the model.

This autoregressive factorization implies that editing the visible prefix changes the context for all subsequent decoding without retraining. Accordingly, Answer Engineering can enforce protocol knowledge through local trajectory edits (Figures 1–3) and primarily targets the failure modes F2–F5.

[2]Implementation detail: after an edit, the KV cache is rebuilt from the earliest modified token. Because the cache is only an efficiency mechanism, rebuilding from the edited prefix yields the same continuation distribution as fresh decoding from that prefix. See Appendix C for a brief implementation note.

4.2 The Trajectory Control Principle

Addresses: F2, F3, F4, F5

Because generation is conditioned only on the visible prefix, a runtime controller can intervene at any point in the reasoning trajectory. These interventions operate directly on the token sequence and can modify previously generated tokens, redirect local continuations, or enforce specific future tokens.

Conceptually, trajectory control allows three classes of intervention, corresponding to the mechanisms illustrated in Figures 1–3.

Editing the past (Figure 1). Previously generated tokens can be replaced when a trigger indicates that a span violates a protocol constraint. The controller identifies a local edit span within the guard scope and substitutes it with a manual replacement candidate chosen according to model likelihood. Because the replacement candidates are defined by the protocol itself, they are assumed to be valid; the controller therefore only selects the highest-likelihood candidate before rebuilding the KV cache and continuing generation.

Selecting among the recently generated continuations (Figure 2). When a trigger appears within the recently generated trajectory, the controller rolls back to the beginning of the edit scope and explores several alternative continuations generated by the model. These continuations are ranked by probability, and invalid ones are rejected according to the protocol rules. The highest-scoring valid continuation is then accepted. If none of the generated continuations satisfy the constraint, the controller falls back to predefined manual candidates.

Forcing the future (Figure 3). Some protocol rules require explicit statements or actions to appear in the reasoning trajectory. Once a trigger condition is detected, the controller selects a predefined future continuation from several manual candidates and appends it after the current prefix. The model is then advanced through the forced tokens, after which normal decoding resumes.

Together, these mechanisms provide a decoding-time control surface for reasoning trajectories.

4.3 Local Intervention versus Global Validation

Addresses: F2, F3, F4, F5

Many reliability approaches rely on *global validation* pipelines, in which a complete answer is first generated

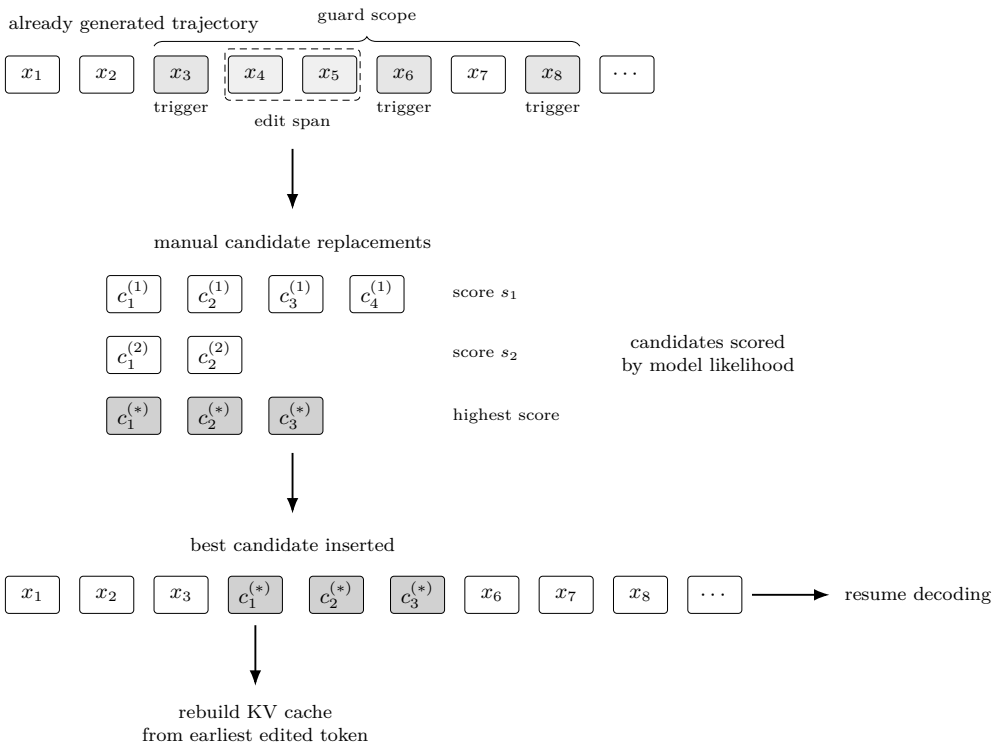

Figure 1: Retroactive span editing. A triggered local span is replaced with the highest-scoring protocol-valid candidate, then decoding resumes from the rebuilt prefix.

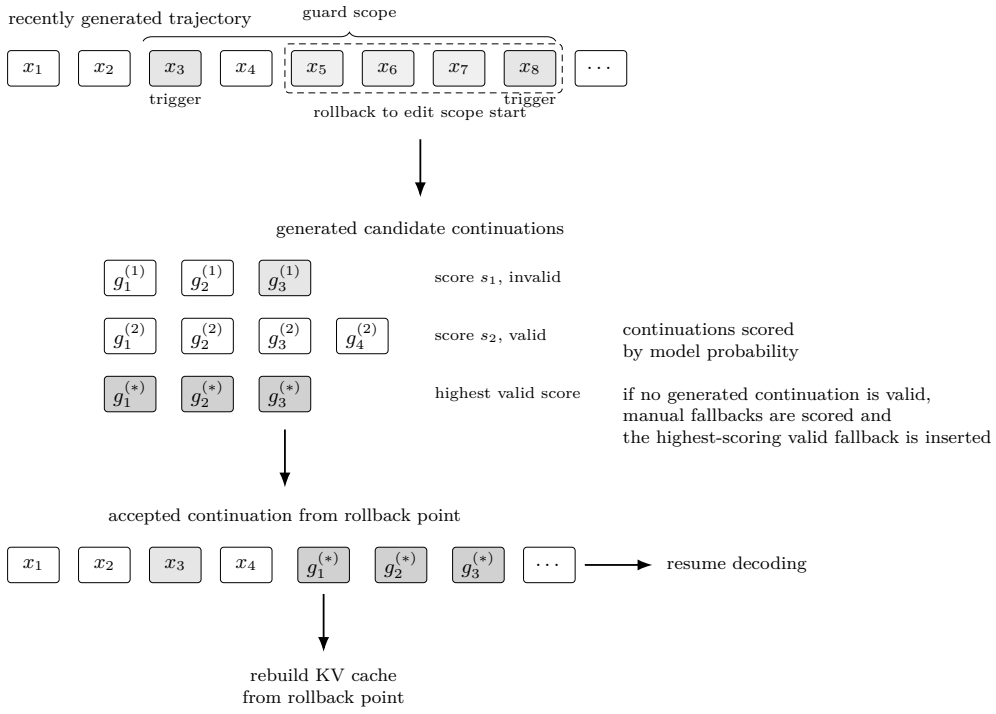

Figure 2: Local rollback and continuation probing. Decoding rolls back to a local scope, evaluates candidate continuations, accepts the highest-scoring valid one (or a fallback), and resumes from the repaired prefix.

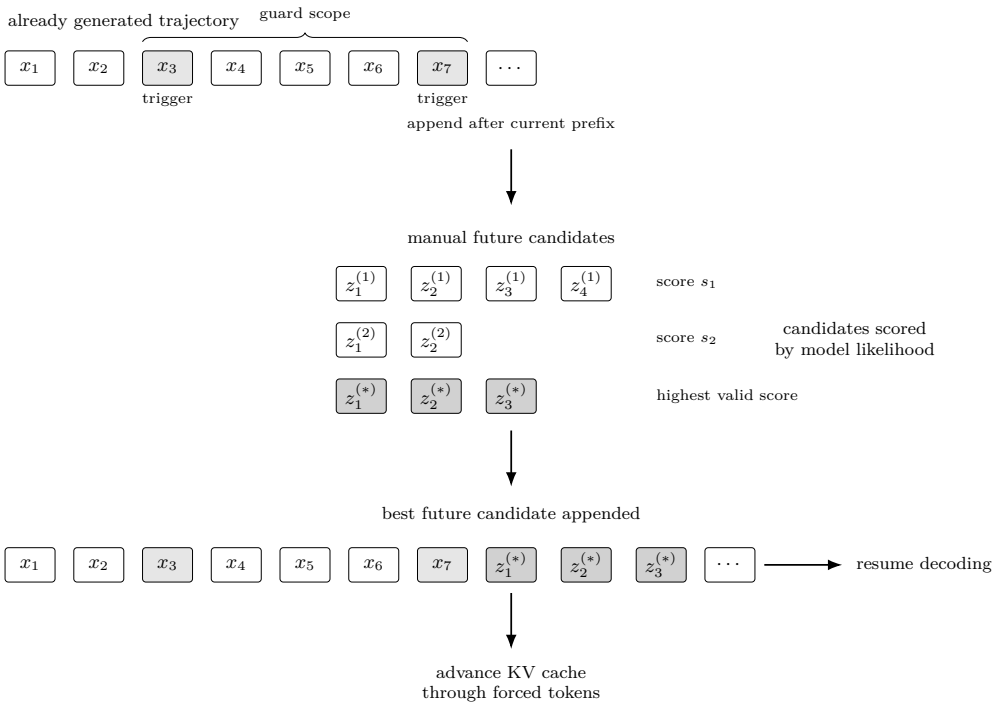

Figure 3: Forced future insertion. When a rule requires a mandatory future statement, the controller appends the highest-scoring valid candidate and continues decoding from the enforced prefix.

and then verified or refined (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024; Miao et al., 2024). Such approaches can substantially improve overall answer quality, but they often require multiple inference passes and may discard largely correct trajectories because of a single local mistake.

Trajectory editing instead follows a *local intervention* strategy: intervene at the violating span and preserve the rest of the trajectory. In this paper, trajectory editing refers to runtime modification of the generated token sequence to enforce protocol constraints before generation continues. The unit of control is a *micro-constraint* on admissible local continuations (representative examples in Appendix B), which is well aligned with failure modes F4–F5.

4.4 Micro-Constraints Instead of Expert Decision Trees

Addresses: F4, F5 (and partially F1)

Traditional rule-based expert systems encode protocols as explicit symbolic rule structures that can specify large portions of the decision process (Reddy and Fields, 2022). While effective in narrow domains, such systems can become difficult to maintain as rule bases grow in size and complexity (Z. Chen and Suen, 1994; Higa, 1996).

Trajectory editing takes a narrower approach than a full expert system. Instead of encoding complete reasoning paths, it represents protocols as local micro-constraints on admissible continuations: forbidding known incorrect inferences, normalizing protocol-critical terminology, or inserting mandatory statements once the relevant concept appears. The system therefore constrains locally dangerous transitions without attempting to encode the entire diagnostic process.

In certain cases, enforcing such constraints can also improve the consistency between intermediate reasoning and final conclusions, partially mitigating unfaithful visible reasoning (F1). For example, if a reasoning trajectory identifies sudden sensorineural hearing loss, a protocol rule may enforce explicit mention of urgent steroid treatment according to established guidelines Chandrasekhar et al. (2019). While this does not guarantee full faithfulness of chain-of-thought reasoning, it ensures that protocol-critical conclusions remain consistent with the stated reasoning.

4.5 Relation to Existing Control Methods

This section's comparison details are summarized in Table 1. In short, trajectory editing differs from constrained decoding, post-hoc verification, and full expert trees by intervening *during decoding* with local trajectory edits scored by model likelihood.

5 Runtime Framework for Answer Engineering

Answer Engineering is implemented as a decoding-time runtime controller that operates alongside standard autoregressive language model inference. The method does not modify model parameters and does not retrain the underlying transformer. Instead, it monitors the evolving generation trajectory, detects rule triggers, proposes candidate trajectory edits, evaluates alternatives under the model likelihood, and applies the selected edit before generation continues.

Let the language model be parameterized by θ with next-token distribution $P_\theta(x_{t+1} \mid x_{1:t})$. Generation therefore proceeds as standard autoregressive decoding, while the Answer Engineering runtime intervenes only when protocol constraints are triggered.

The intervention mechanisms themselves were introduced conceptually in Section 4.2 and illustrated in Figures 1–3. The present section describes how these interventions are executed during decoding. Implementation details and reproducibility notes are summarized in Appendix C. The rule language used to express protocol constraints is summarized in Appendix A, with representative examples in Appendix B.

5.1 Rule representation

Protocol constraints are expressed as declarative rules. Each rule specifies

- a **trigger pattern** that detects a trajectory condition,
- a **scope** identifying the region of the trajectory affected,
- a set of **candidate edits**.

Candidate edits represent alternative modifications of the trajectory once the trigger fires. These edits may include

- manually authored protocol phrases (e.g. required management statements),
- locally generated alternative continuations proposed by the model, or
- structured edits such as span replacement, rollback continuation, or token insertion.

Rules therefore encode *local trajectory constraints* rather than complete reasoning programs. The model remains responsible for generating the reasoning trajectory, while rules modify only those local segments of the trajectory that violate protocol constraints.

Rules are authored in a compact Markdown-based domain-specific language that domain experts can inspect and modify without modifying model code. During initialization, rules are parsed and compiled into executable runtime plans used during decoding.

Trajectory edit operator. Let the current trajectory be $T = x_{1:t}$. A candidate intervention produces an edited trajectory

$$T' = \mathcal{E}(T; i, j, c) = x_{1:i-1} \,\|\, c \,\|\, x_{j+1:t},$$

where $[i, j]$ denotes the affected span and c is the candidate replacement sequence. The edit may correspond to

- retroactive span correction,
- rollback continuation replacement, or
- insertion of a required future statement.

Subsequent decoding then proceeds from the edited prefix T' under the same autoregressive model P_θ.

5.2 Runtime decoding procedure

Generation proceeds autoregressively while the runtime monitors the trajectory for rule triggers. Whenever a trigger fires, the controller evaluates candidate trajectory edits and applies the highest-scoring compatible intervention before decoding continues.

All reported experimental configurations use deterministic greedy decoding. Thus, repeated runs with the same model, dataset, rule set, and environment produce the same generated trajectories.

Runtime is reported as the mean per-case wall-clock time measured over the full evaluation run, measured from the start of model decoding after prompt construction. This measurement includes model generation, rule evaluation, intervention handling, KV-cache reconstruction, and telemetry construction.

Slowdown is defined as average seconds per case relative to the corresponding reasoning-generation run within the same diagnostic scope. Values less

than 1 indicate faster execution, which corresponds to acceleration rather than degradation.

Algorithm 1 formalizes the runtime step procedure.

Algorithm 1 Answer Engineering Runtime Decoding

Require: model P_θ, prompt $x_{1:n}$, rule plans Π
$T \leftarrow x_{1:n}$
while generation not finished **do**
  sample $x_{t+1} \sim P_\theta(\cdot \mid x_{1:t})$
  append x_{t+1} to T
  $\mathcal{G} \leftarrow$ detect_triggers(T, Π)
  if $\mathcal{G} \neq \emptyset$ **then**
    bucket $\mathcal{G}$ by earliest edit position
    for buckets b in ascending edit position **do**
      $\mathcal{C} \leftarrow$ propose_candidates(b, T)
      $\mathcal{A} \leftarrow$ filter_admissible$(\mathcal{C}, \Pi)$
      if $\mathcal{A} \neq \emptyset$ **then**
        let $i(c)$ be the first affected token position of c
        compute $s(c; T) = \log P_\theta(c \mid x_{1:i(c)-1})$
        $\hat{\mathcal{E}} \leftarrow$ select_compatible$(\mathcal{A}, s)$
        resolve overlaps in bucket
        apply $\hat{\mathcal{E}}$ to T deterministically
        $p \leftarrow$ earliest modified token
        rebuild KV cache from p
        break
      end if
    end for
  end if
end while
return T

The runtime therefore acts as a lightweight trajectory controller embedded inside standard transformer decoding.

Earliest-bucket execution. Triggered rules may affect different portions of the trajectory. To ensure consistent edits, rules are grouped into *edit buckets* according to the earliest token position they modify. Buckets are processed in ascending order of edit position.

Only the earliest bucket that yields admissible edits is executed during a given decoding step. Later buckets are deferred until generation resumes from the updated trajectory. This ordering prevents edits from being evaluated on a trajectory prefix that may subsequently change.

Candidate generation and rollback probing. When a bucket is selected, the runtime constructs candidate edits consistent with the triggered rule semantics. Candidate edits may be

- explicit protocol statements provided by the rule definition,
- model-generated rollback continuations,
- structured edits such as span replacement or insertion.

Rollback candidates are generated using a deterministic beam-style probe procedure applied to the rollback prefix. In the current implementation, candidate generation uses a shared beam-search routine with optional grouped-beam diversification and diversity penalties to encourage distinct continuations before protocol validity filtering.

Candidate scoring. Each admissible candidate edit c is scored using the model likelihood under the prefix preceding its first affected token position $i(c)$:

$$s(c; T) = \log P_\theta(c \mid x_{1:i(c)-1}).$$

For insertions after the current prefix, $i(c) = t + 1$, so this reduces to $\log P_\theta(c \mid T)$. This convention makes explicit that replacement candidates are scored against the prefix before the affected span, not against the span being replaced. Using the model's own scoring function ensures that injected protocol phrases remain linguistically coherent with the surrounding reasoning.

Conflict resolution and deterministic application. Multiple candidate edits may target overlapping spans. The runtime therefore performs conflict resolution before applying edits. Compatible edits within the active bucket are selected according to their model scores and applied in deterministic order.

KV-cache rebuild after edits. After a trajectory edit, decoding resumes from the edited prefix by rebuilding the KV cache from the earliest modified token. Since the cache is an efficiency device rather than an independent state, this yields the same continuation distribution as standard decoding from the edited prefix, up to normal numerical and sampling variance.

External protocol memory. Protocol knowledge is maintained externally rather than embedded inside model parameters. When a rule fires, protocol statements or structured knowledge fragments can be injected directly into the trajectory as candidate edits. Because transformer decoding conditions only on the visible prefix, these inserted tokens immediately become part of the model's effective context.

5.3 Intervention mechanisms

During decoding the runtime continuously monitors the evolving trajectory for rule triggers. When a

trigger fires, candidate edits are proposed, evaluated under the model likelihood, and the selected intervention is applied before generation resumes.

These runtime interventions implement the three trajectory-control mechanisms introduced in Section 4.2:

- **editing the past** (retroactive span replacement),
- **selecting among recent continuations** (local rollback with beam-style probing),
- **forcing the future** (insertion of required protocol statements).

Algorithm 1 specifies the runtime execution semantics, while Figure 4 illustrates the same process as a conceptual control loop.

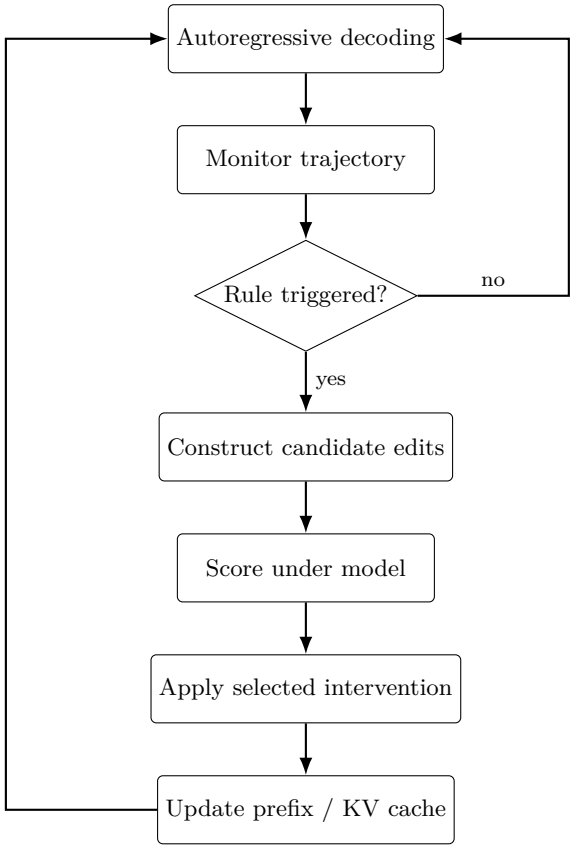

Figure 4: Conceptual runtime control loop. Autoregressive decoding produces tokens while the runtime monitors the trajectory for rule triggers. When a rule fires, candidate trajectory edits are generated (possibly via beam-style probing), evaluated under the model likelihood, and the selected intervention is applied before decoding continues from the modified prefix.

6 Experimental Demonstration: Sudden Sensorineural Hearing Loss (SSNHL)

We evaluate trajectory editing on a diagnostic reasoning task involving sudden sensorineural hearing loss (SSNHL). The task requires combining multiple clinical cues and exposes common language-model failure modes such as inconsistent interpretation of diagnostic tests, premature conclusions, and omission of mandatory protocol steps.

All experiments used the publicly released medical language model `OpenMeditron/Meditron3-8B` (revision `15914bcb040cd1a4f263afcd85b84f09ad2efd95`) (OpenMeditron, 2025). No local model modifications were applied. All evaluations were executed with deterministic decoding settings, ensuring that differences between configurations reflect intervention behavior rather than sampling variability. Trajectory editing was applied during decoding using the runtime control loop in Figure 4.

The goal of this experiment is to test feasibility of decoding-time trajectory control on a narrow, protocol-sensitive benchmark rather than to model the full complexity of clinical otologic decision-making.

6.1 Clinical background

Sudden sensorineural hearing loss (SSNHL) is a time-sensitive clinical condition requiring prompt evaluation.[3] In this benchmark, diagnosis depends on combining symptom timing with bedside Weber and Rinne tuning-fork findings under standard AAO–HNS interpretation rules.[4]

6.2 Dataset

We evaluate on a public deterministic dataset of synthetic, parameterized clinical vignettes designed to isolate a protocol-sensitive decision boundary while preserving a clinically plausible presentation structure. The design intentionally uses symmetric paired templates for SSNHL-consistent and conductive cases: age, laterality, time since onset, distractor details, and surface wording vary, while the decisive Weber/Rinne and otoscopic findings are changed in a class-consistent way. For evaluation, we use two balanced subsets: $N = 1000$ SSNHL-consistent cases and $N = 1000$

[3] Clinically, SSNHL is typically defined as a sensorineural hearing loss of at least 30 dB affecting three consecutive frequencies and developing within 72 hours (Chandrasekhar et al., 2019).

[4] Appendix E reproduces the benchmark-facing interpretation summary used for Weber and Rinne testing.

contrastive conductive cases. Each case combines symptom timing, Weber/Rinne findings, and class-aligned otoscopic examination, with secondary distractor details included but not made decisive. This benchmark is used for controlled relative assessment of reasoning trajectories rather than as a surrogate for full-spectrum clinical performance or natural clinical prevalence, and the detailed template design rationale is provided in Appendix E.

6.3 Evaluation criterion

For benchmarking purposes, protocol adherence is operationalized as a binary endpoint: whether the answer explicitly recommends steroid therapy. This endpoint captures the key treatment branch distinguishing suspected SSNHL from conductive hearing loss.[5] For SSNHL cases, answers that explicitly recommend steroid therapy satisfy the benchmark criterion; for conductive cases, they do not. This endpoint is intentionally narrow and isolates the key therapeutic branch in the protocol, testing adherence to a decisive management step rather than full clinical answer quality.

Each generated case is treated as an independent sample in metric computation.

Figure 5 illustrates how trajectory revision can improve protocol-consistent reasoning and management.

6.4 Overall results

Table 3 summarizes performance across progressively more localized intervention mechanisms applied to the generation process. The evaluated configurations range from prompt-level adjustments to trajectory-level coordination of validation and repair.

Rules were developed using the training split of the publicly available dataset. A separate held-out test split was defined but was not used during rule development. The results reported in this paper correspond to the training split, which serves as a controlled benchmark for evaluating runtime intervention behavior.

The rule set was finalized prior to executing the reported evaluation, and no rule modifications were made during metric computation.

Baseline LLM The unguided baseline exhibits a strong diagnostic bias toward SSNHL treatment. In this configuration, the model frequently recommends steroid therapy regardless of the clinical presentation, producing moderate acceptance rates on SSNHL cases but near-zero acceptance on conductive contrast cases. This pattern indicates a prior-dominated response strategy rather than reliable differential reasoning, and results in low balanced accuracy.

Reasoning Introducing step-by-step reasoning changes the error pattern but does not resolve the underlying instability. Compared with the unguided baseline, the reasoning configuration improves acceptance on conductive cases but reduces acceptance on true SSNHL cases. Balanced accuracy increases relative to the baseline, but the improvement reflects a redistribution of errors rather than consistent protocol compliance. The model becomes less biased toward SSNHL but more likely to misclassify true SSNHL cases as conductive hearing loss.

Global validation Global validation is a widely used constraint-enforcement strategy in sequence generation systems, in which multiple candidate responses are generated and filtered against protocol requirements before final selection Hokamp and Q. Liu, 2017; Zhang et al., 2023; Koo et al., 2024. In the present evaluation, alternative responses are produced using group beam search to explore diverse reasoning trajectories during decoding Vijayakumar et al., 2018. Candidate outputs that violate critical constraints are rejected or replaced with compliant alternatives.

This mechanism improves reliability relative to the reasoning configuration by reducing the number of unacceptable decisions while preserving the underlying generation process. Because validation operates at the level of completed candidate outputs, the reasoning trajectory itself is not modified. Performance improvements therefore arise from selecting compliant alternatives rather than from changing the model's diagnostic bias during generation.

Local validation Local validation extends this validation paradigm by applying the same constraint checks earlier in the generation process, monitoring reasoning steps as they are produced rather than evaluating only completed responses. Instead of generating full alternative outputs and selecting among them, local validation detects protocol violations at intermediate points in the trajectory and can trigger corrective actions without regenerating the entire response.

In the present evaluation, this earlier detection does not produce a substantial shift in diagnostic acceptance rates relative to global validation, indicating that the timing of validation alone does not materially alter the underlying decision bias of the

[5] Appendix E explains the endpoint choice and contrasts it with alternative candidate signals.

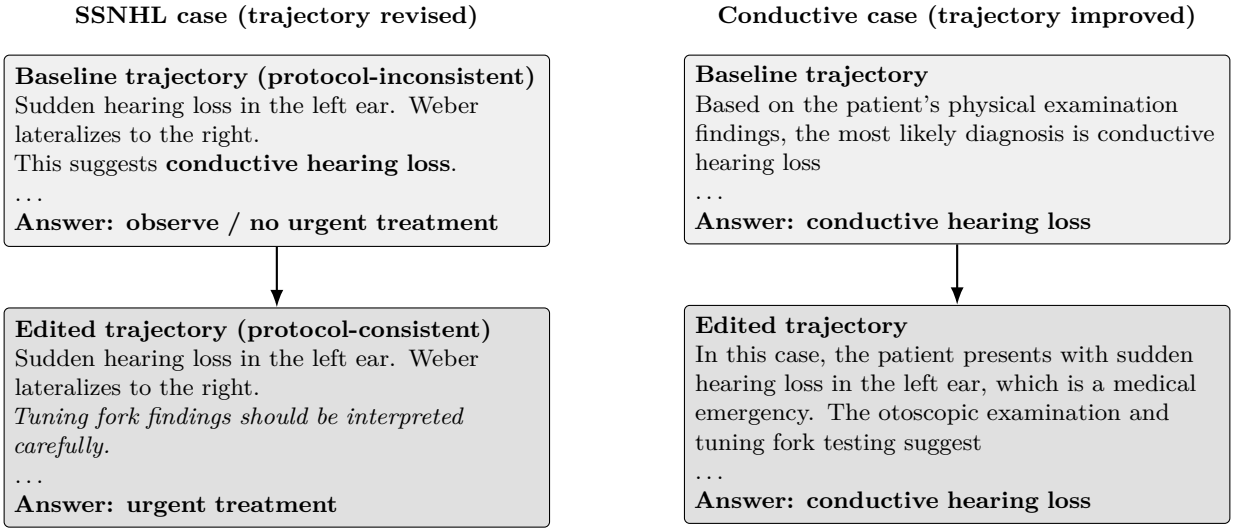

Figure 5: Two representative trajectories.
Left: trajectory editing revises an SSNHL case by enforcing protocol-consistent interpretation of tuning fork findings, leading to protocol-consistent management.
Right: in conductive hearing loss cases, trajectory editing can also improve reasoning fidelity to the stem, preserving the conductive diagnosis while reducing protocol-inconsistent detours.

	SSNHL		Conductive		Balanced accuracy
Method	Accepted	Slowdown	Accepted	Slowdown	
Baseline LLM	54.5%	0.31×	1.6%	0.34×	28.1%
Reasoning	25.1%	1.00×	58.9%	1.00×	42.0%
Global validation	39.5%	4.49×	–	–	–
Local validation	40.3%	3.15×	–	–	–
Replace + After only	33.7%	0.96×	–	–	–
Trajectory editing	83.5%	2.78×	77.9%	1.33×	80.7%

Table 3: Performance on SSNHL ($N = 1000$) and contrastive conductive cases ($N = 1000$). Slowdown is reported relative to the reasoning run within the same diagnostic scope; values below 1× indicate acceleration. Balanced accuracy averages the SSNHL and conductive acceptance rates when both scopes are available. Trajectory editing combines local validation with Replace and After rules into a coordinated intervention mechanism.

model. However, local validation provides a clear operational advantage: violations can be identified and addressed incrementally, resulting in reduced regeneration overhead and improved runtime efficiency compared with output-level validation.

Manual Replace + After only Replace and After interventions provide a direct-edit alternative to validation-based selection. Instead of generating multiple candidate trajectories and filtering them against protocol constraints, these rules enforce manually authored protocol-compatible trajectory fragments at specific trigger points. When several candidate fragments are available, they are scored under the model likelihood, and the highest-likelihood compatible fragment is inserted so that the edited trajectory remains linguistically coherent with the surrounding generation.

This mechanism can correct isolated protocol failures without regenerating or selecting among full candidate responses. However, when applied without local validation, edits are limited to predefined trigger–repair patterns and do not provide a general mechanism for detecting and resolving arbitrary local violations during generation.

Trajectory editing (Local validation + Replace + After) Trajectory editing combines local validation with coordinated Replace and After rules to detect, repair, and propagate corrections during generation. In this configuration, acceptance rates increase simultaneously in both diagnostic scopes

rather than shifting between them. Balanced accuracy rises substantially relative to all baseline and intermediate configurations, indicating that coordinated local correction of reasoning steps can improve decision quality without introducing compensatory degradation in other task variants.

Importantly, the applied rules were designed to enforce faithful protocol-compliant reasoning rather than to promote a specific diagnosis. For example, early diagnostic conclusions were prohibited uniformly, including premature inference of both conductive hearing loss and SSNHL. All trajectories were required to follow the same reasoning sequence, beginning with structured test interpretation and proceeding to diagnosis only after required evidence had been evaluated. This symmetric enforcement ensures that improvements do not arise from suppressing alternative diagnostic pathways or artificially increasing the relative probability of one class.

Under these constraints, improvements observed in both SSNHL and contrastive conductive cases are consistent with stabilization of the reasoning process rather than simple redistribution of diagnostic bias. Enforcing consistent reasoning order is consistent with reduced premature conclusions and improved alignment between intermediate reasoning steps and final decisions across task variants.

Across configurations, performance improves as intervention mechanisms become more localized and coordinated. This pattern is consistent with many observed incorrect decisions are compatible with premature or inconsistent reasoning trajectories rather than missing medical knowledge alone. In this controlled benchmark, stabilizing these trajectories during decoding restores protocol-consistent outcomes in a large fraction of cases.

6.5 Runtime telemetry and intervention behavior

Tables 4 and 5 summarize trajectory-editing behavior on the SSNHL subset only ($N = 1000$).

Avoid-rule resolution accounts for the majority of accepted edits in this run. Alternative trajectory generation introduces additional decoding work but is typically resolved after evaluating only a small number of candidate continuations before generation resumes.

Table 5 characterizes the search depth required to resolve avoid-rule episodes. Percentile values indicate the minimum number of alternative trajectories required to resolve the specified fraction of episodes. The “episodes exhausting alternative budget” metric identifies episodes in which the predefined alternative budget was fully consumed[6]

All telemetry metrics are computed directly from structured runtime logs using the same instrumentation used to generate the reported evaluation tables, ensuring consistency between behavioral telemetry and outcome metrics.

Metric	Value
"Avoid" interventions per case	15.4
All interventions per case	17.1

Table 4: Intervention frequency statistics computed over all evaluated cases on the SSNHL subset ($N = 1000$). Values are averaged across all cases, including cases with zero interventions.

Metric	Value
Avgerage alternatives tried	3.70
Alternatives for 50% resolution	2
Alternatives for 80% resolution	6
Episodes exhausting alternative budget	8.7%

Table 5: Alternative-search behavior during avoid-rule resolution on the SSNHL subset ($N = 1000$). Percentile rows indicate the minimum number of alternative trajectories required to resolve the specified fraction of avoid-rule episodes.

6.6 Edit trace examples

Table 6 provides representative edit traces from runtime logs, including protocol-relevant interpretations of Weber and Rinne findings and the corresponding management statement.

6.7 Paired trajectory changes across target and contrastive tasks

The conductive task serves as a contrastive control condition for evaluating unintended side effects of protocol enforcement while also providing an independent signal about trajectory behavior outside the target protocol.

Table 7 summarizes the paired changes induced by the different intervention bundles across both tasks. The SSNHL rows show that the full trajectory-editing stack delivers the largest net gain on the target protocol. In the conductive setting, trajectory editing

[6]Budget exhaustion does not indicate failure to produce a final answer. In such cases, generation continues using the highest-scoring available trajectory after the alternative search budget is reached.

Baseline fragment	Rule trigger	Edited fragment
"The patient presents with sudden hearing loss in the **left** ear, which is a concerning symptom. The **Weber** test lateralizing to the **right** ear suggests that the left ear has a ~~**conductive**~~"	Contralateral Weber lateralization invalidates conductive inference	"The patient presents with sudden hearing loss in the left ear, which is a concerning symptom. The Weber test lateralizes to the right ear*, indicating that the left ear is worse, and the Rinne test is positive on the left ear...*"
"... and the **Rinne** test is **positive** on the left ear, suggesting that the hearing loss is ~~**conductive**~~"	Positive Rinne invalidates conductive inference	"..., and the Rinne test is positive on the left ear*, indicating that air conduction is better than bone conduction...*"
"The patient presents with **sudden** hearing loss ... These findings are consistent with **sensorineural hearing loss**"	Enforce "sudden" qualifier for sensorineural hearing loss	"The patient presents with sudden hearing loss ... These findings are consistent with ***sudden*** sensorineural hearing loss..."
"... These findings are consistent with **sudden sensorineural hearing loss**"	Add urgent treatment statement for SSNHL	"... These findings are consistent with sudden sensorineural hearing loss. *This condition requires urgent treatment...*"

Table 6: Representative edit traces extracted from runtime logs.

Method	Scope	Improved	Degraded	Δ acc.
Global validation	SSNHL	308	164	+14.4 pp
Local validation	SSNHL	272	120	+15.2 pp
Replace + After only	SSNHL	88	2	+8.6 pp
Trajectory editing	SSNHL	610	26	+58.4 pp
Trajectory editing	Conductive	200	10	+19.0 pp

Table 7: Paired change summary across SSNHL and conductive runs, reported in raw counts.

also produces a substantial positive shift (+19.0 pp), with improvements (+20.0 pp) exceeding degradations (+1.0 pp).

This pattern indicates that the intervention mechanism is performing genuine trajectory modification rather than merely enforcing target-specific labels. Local trajectory interventions appear to improve not only protocol compliance on the SSNHL task but also the stability and clinical alignment of reasoning in cases governed by a different diagnostic pathway.

At the same time, the result highlights an important interpretive limitation: the current evaluation does not fully disentangle gains arising from task-specific protocol repair versus gains arising from more general trajectory stabilization. Clarifying that distinction, and improving trigger specificity where unnecessary edits remain, is an important direction for future work.

6.8 Model Sensitivity and Baseline Pathologies

Although all formal results in this study use OpenMeditron/Meditron3-8B, we conducted exploratory pilot runs on additional medical-language models to assess how trajectory-level control behaves under different baseline reasoning characteristics. In particular, we tested OpenMeditron/Meditron3-Qwen2.5-7B and HPAI-BSC/Llama3.1-Aloe-Beta-8B. These pilots were performed on small samples far smaller than the primary benchmark and were intended for qualitative inspection rather than statistical comparison. For this reason, we report behavioral patterns rather than specific percentages.

The Qwen-based model was substantially slower in our setup but exhibited the same qualitative improvement pattern as the primary model: local interventions and full trajectory editing improved reasoning outcomes relative to baseline. This observation suggests that the core mechanism of trajectory-level

intervention transfers across related architectures when the model maintains interpretable intermediate reasoning steps.

In contrast, the Aloe model exhibited a baseline pathology rather than a trajectory-level failure. Before any intervention, it frequently defaulted to an SSNHL diagnosis across diverse presentations, including conductive cases. Inspection of generated reasoning traces showed that the model often misread Weber/Rinne findings early in the reasoning process and committed immediately to the emergency diagnosis regardless of case-specific evidence. As a result, the baseline appeared highly accurate for SSNHL cases but performed poorly on contrastive conductive cases.

This behavior indicates that some models may enter evaluation with strongly skewed diagnostic priors or limited reasoning fidelity, producing superficially high performance on a subset of tasks while failing to perform meaningful differential diagnosis.

6.9 Large Gains and Trajectory Instability in Weak Models

Given the baseline pathology described in Section 6.8, in which the Aloe model frequently defaulted to an SSNHL diagnosis regardless of case-specific evidence, we constructed a rule configuration in which conductive hearing loss served as the primary condition while SSNHL remained the contrastive case. In this setting, trajectory editing produced substantial improvements relative to baseline, often increasing accuracy by several-fold. These gains were qualitatively similar to the main experiments but were more pronounced because the initial model performance was substantially lower.

However, the intervention dynamics differed from those observed with the primary model. Because the baseline reasoning trajectory frequently committed early to an incorrect diagnosis, redirecting the trajectory often required repeated fallback corrections within the same generation. The frequency of such corrections was substantially higher than in the primary experiments, where fallback interventions occurred only intermittently.

When fallback density became high, the resulting responses sometimes exhibited unstable reasoning structure, including excessive verbosity, repetition, or internally inconsistent phrasing. These effects did not typically alter the final protocol-compliant decision but indicated that the model had been driven into low-probability internal states through repeated trajectory redirection. In other words, large improvements in final-answer accuracy did not necessarily correspond to improvements in reasoning quality or stability.

This observation suggests that trajectory-level intervention remains capable of substantially improving accuracy even for weak or poorly calibrated models, but that the efficiency and robustness of the resulting reasoning depend on the base model's ability to sustain coherent trajectories under repeated correction.

More broadly, these findings motivate a separate line of investigation into the faithfulness and controllability of model reasoning processes. In particular, future work should examine how readily a model can diverge from an incorrect early diagnosis, whether alternative reasoning trajectories remain accessible during generation, and how diverse candidate continuations can be produced at critical pivot points. Such studies may involve systematic comparison of probe generation strategies, including approaches that explicitly encourage trajectory diversity (e.g., self-training or iterative refinement methods that generate diverse reasoning paths, such as STaR (Zelikman et al., 2022) or self-consistency decoding (Wang et al., 2023)), and may provide practical criteria for selecting models that are most responsive to trajectory-level intervention.

7 Limitations

The present evaluation is conducted within a controlled, single-protocol diagnostic boundary designed to isolate specific reasoning failures. The benchmark constrains the decision space to a narrow causal structure in which protocol violations are explicitly observable. The results therefore establish mechanistic feasibility under structured conditions rather than general performance across heterogeneous workflows or interacting protocols.

Performance depends on the correctness and completeness of the authored rule set. Trajectory editing externalizes domain knowledge into explicit repair policies, and observed improvements may partially reflect the validity of those policies rather than intrinsic changes in model reasoning capability. As protocol complexity increases, rule coverage and validation requirements scale accordingly.

The evaluation focuses on controlled benchmark feasibility rather than generalization to unseen data.

The dataset consists of structured variants of symmetric template families rather than statistically independent real-world patient cases. This design is useful for isolating a protocol-sensitive decision boundary, but it limits claims about natural clinical prevalence, distributional diversity, and real-world case heterogeneity.

Trajectory editing improves decision outcomes but does not guarantee uniform improvements in reasoning stability. In models with unstable or poorly calibrated

reasoning dynamics, repeated local corrections may increase generation length, latency, or verbosity. The method therefore assumes the existence of locally recoverable reasoning trajectories that can be incrementally repaired during decoding.

System behavior depends on reliable detection of protocol-relevant triggers. False positives may introduce unnecessary edits, while false negatives may allow violations to pass uncorrected. Trigger calibration therefore directly affects intervention frequency, computational cost, and outcome reliability.

Evaluation is limited to a small set of model architectures and decoding configurations. Differences in prior distributions, verbosity patterns, or reasoning stability may influence both intervention frequency and repair cost. Operational characteristics should therefore be interpreted as model-dependent.

Improved protocol adherence does not imply improved real-world performance. The evaluation measures conformity to procedural requirements rather than downstream clinical or operational outcomes. External validation in realistic decision environments remains necessary.

Generalization to new domains depends on the presence of explicit procedural structure and locally correctable reasoning errors. Tasks lacking stable protocol boundaries or interpretable intervention points may require alternative control mechanisms.

8 Conclusion

This work presents Answer Engineering, a runtime framework for improving protocol compliance in large language model reasoning through localized trajectory control. Instead of modifying model parameters, the framework applies explicit rule-based interventions directly to generated reasoning traces, enforcing required procedural steps at critical decision points.

In the controlled diagnostic benchmark, trajectory-level interventions increased balanced diagnostic accuracy from 42.0% under reasoning-only generation to 80.7% with trajectory editing. The gains were observed in both diagnostic scopes: +58.4 pp on the SSNHL task and +19.0 pp on the conductive contrast task. These improvements were achieved without retraining the underlying model, with runtime overhead of 2.78× on SSNHL cases and 1.33× on conductive cases relative to the corresponding reasoning baselines.

The results should be interpreted as evidence of feasibility under structured protocol conditions rather than as a general solution to reasoning reliability. The current evaluation isolates a well-defined diagnostic boundary in order to make reasoning failures observable and correctable through explicit rules. Real-world workflows frequently involve interacting protocols, partial observability, and evolving guidelines, all of which introduce additional sources of uncertainty beyond the scope of the present experiments.

Within these constraints, the findings support a systems-level perspective in which protocol adherence can be engineered operationally through explicit control of generation behavior. Answer Engineering demonstrates that reliability in protocol-governed reasoning is not solely a property of model training, but can also be treated as an operational property shaped by runtime control of reasoning trajectories.

9 Future Work

Future work should evaluate whether trajectory-level control can scale from localized protocol repair to broader runtime reliability mechanisms.

We organize this agenda as a set of testable research objectives rather than as a list of system features. Each direction is motivated by known limitations of current reasoning and constrained-generation methods, and is framed in terms of expected behavioral changes that can be empirically measured.

9.1 Cross-domain protocol generalization

Motivation. The present evaluation isolates a single clinical protocol boundary in order to make reasoning failures observable and correctable. However, protocol-constrained generation is not specific to medicine. Similar failure modes have been observed in legal compliance, financial decision support, software configuration, and regulated communication workflows, where outputs must satisfy externally defined procedures rather than maximize fluency alone. Prior work on structured reasoning, constrained decoding, and guideline adherence demonstrates that models can follow explicit instructions while still violating domain-specific protocols under distribution shift or ambiguity.

Expected result. We expect trajectory editing to produce the largest gains in domains where errors are localized, protocol rules are explicit, and intermediate reasoning remains observable. We do not expect uniform improvements in domains where failures primarily reflect missing external knowledge or ambiguous standards. Instead, we expect performance improvements to correlate with the clarity and stability of the governing protocol.

Evaluation. Future studies should measure protocol-compliant outcome rate, balanced accuracy across target and contrastive cases, degradation rate on control tasks, edit frequency per case, and rule coverage across domains. A successful result would demonstrate improved protocol adherence without simply shifting errors between paired task variants.

9.2 Causal trajectory repair

Motivation. The current runtime primarily resolves violations at or near the point where they are detected. In longer reasoning trajectories, however, downstream errors often originate from earlier incorrect commitments. Prior analyses of reasoning behavior have shown that generated explanations may rationalize an answer rather than derive it step by step. In such cases, repeated frontier-level correction may be less effective than repairing the earliest causal error.

Expected result. We expect causal repair to reduce repeated interventions, improve trajectory stability, and increase the probability that downstream reasoning remains consistent after correction. The central hypothesis is that repairing the earliest incorrect commitment will produce more coherent reasoning than repeatedly suppressing later symptoms.

Evaluation. This direction should be evaluated using intervention density, number of repeated violations after repair, trajectory coherence measures, final protocol adherence, and preservation of valid downstream reasoning. Controlled ablations should compare local frontier repair against upstream causal-span repair under identical rule sets.

9.3 Branch-aware trajectory control and uncertainty representation

Motivation. The present system selects a single accepted trajectory after each intervention. However, many decision problems admit multiple plausible continuations. Existing methods such as self-consistency and multi-sample reasoning improve accuracy by evaluating multiple reasoning paths, but typically collapse alternatives into a single final answer without preserving the structure of competing trajectories. In safety-critical settings, the presence of multiple valid interpretations may itself be operationally significant.

Expected result. We expect branch-aware control to improve transparency and calibration in cases where multiple valid reasoning paths exist. Rather than presenting a single answer as uniquely determined, the system could record when materially different valid trajectories were available and surface that condition explicitly.

Evaluation. Future work should measure branch diversity, agreement between branches, branch-level protocol compliance, calibration of uncertainty signals, and the frequency with which branch-aware control changes the final decision. In high-stakes environments, success should be defined not only by accuracy but also by whether the system correctly signals unresolved ambiguity.

9.4 Adaptive intervention policies and rule lifecycle management

Motivation. Trajectory editing introduces computational overhead relative to unconstrained generation and requires explicit rule authoring by domain experts. Production deployment therefore requires mechanisms that balance reliability, latency, and maintainability. Prior work on constrained decoding and guardrail systems shows that safety interventions must be selectively applied to remain operationally viable at scale.

Expected result. We expect adaptive intervention policies to reduce unnecessary edits and alternative-search cost while preserving most compliance gains. We also expect structured rule lifecycle tooling to reduce the effort required to extend trajectory editing to new protocols.

Evaluation. Efficiency should be measured using latency, slowdown relative to reasoning-only generation, alternatives tried per avoidance episode, cache rebuild frequency, and accepted edits per case. Rule lifecycle performance should be evaluated through coverage analysis, regression testing across protocol versions, and expert time required to author or update rule sets.

9.5 Rule discovery and authoring assistance

Motivation. The present system relies on manually authored rules created from expert protocol knowledge and observed failure patterns. This keeps interventions auditable, but it may limit scalability when protocols are large, frequently updated, or difficult to encode exhaustively.

Expected result. We expect rule-discovery tools to reduce expert authoring effort by proposing candidate triggers, common violation patterns, and candidate repairs from trajectory logs. Such tools should assist expert review rather than replace it, because protocol rules remain safety-critical artifacts.

Evaluation. This direction should be evaluated by measuring expert authoring time, rule coverage, false-positive and false-negative trigger rates, regression failures after rule updates, and the fraction of automatically proposed rules accepted by domain experts.

9.6 Summary

Across these directions, the central research question is whether reliability can be engineered as an operational property of generation rather than treated only as a property of model training. The expected outcome is not a replacement for model improvement, expert review, or external validation, but a complementary runtime layer that makes protocol violations more observable, correctable, and auditable during generation.

10 Reproducibility

All code, configuration files, datasets, and notebooks required to reproduce the experiments are publicly available at:

https://github.com/victorlavrenko/answer-engineering

The main experiments can be reproduced using the Google Colab notebook:

notebooks/reproduce.ipynb

Experiments were executed in Google Colab on a G4 GPU runtime. In this runtime, `nvidia-smi` identified the provisioned accelerator as an NVIDIA RTX PRO 6000 Blackwell GPU with approximately 96 GB VRAM.@

All reported runs were executed in this Colab environment using deterministic greedy decoding.

Reproduction may require additional configuration, including setting authentication secrets (e.g., Hugging Face tokens), accepting access conditions for gated models, and optionally configuring telemetry publication. Detailed step-by-step instructions, environment setup requirements, and artifact descriptions are provided in:

docs/current/reproducibility.md

All numerical results reported in this paper are generated automatically from evaluation outputs into a machine-readable TEX file:

docs/paper/generated/paper-metrics.tex

The experiments use `OpenMeditron/Meditron3-8B` from Hugging Face. The reproduction manifest records the model/config used by the run; however, the all reported results were generated using a single fixed model revision.

Appendix overview. The appendices provide concise summaries for reviewers, including a compact overview of the rule language (Appendix A), representative rule targets (Appendix B), a summary of the reproduction pipeline and generated artifacts (Appendix C), representative evaluation prompts (Appendix D), and the clinical justification of the benchmark endpoint (Appendix E).

Appendix A: Compact rule language overview

The Answer Engineering rule language is a Markdown-based interface for authoring trajectory-editing constraints that domain experts can review and modify without changing the model code. The rules are parsed into a structured representation and compiled into executable rule plans used during generation.

Importantly, rules do not inject diagnoses or replace the model's reasoning with expert-system logic. Instead, they operate as local trajectory constraints: they invalidate known incorrect reasoning continuations, normalize terminology, or enforce protocol-compliance statements when a relevant concept has already been generated.

The rule language is organized into several families corresponding to common trajectory-editing operations. These families are summarized in Table 8.

For example, the following rule prevents a common reasoning error where contralateral Weber lateralization is incorrectly interpreted as conductive hearing loss.

```
## Avoid: contralateral conductive Weber
Prefix:
* Weber | forehead
* left || right

Postfix:
* right || left
```

Family	Purpose
`Replace`	Normalize protocol-critical terminology without changing the underlying reasoning.
`After`	Insert protocol-required statements once a concept has already been produced.
`Avoid`	Invalidate known incorrect reasoning continuations and redirect generation.
`Force`	Guarantee required protocol elements appear when applicable.

Table 8: Rule families used to constrain reasoning trajectories during generation.

```
* conductive

Fallback:
* The Weber finding should be interpreted
  in relation to the affected ear.
```

The rule prevents a specific incorrect inference and redirects the reasoning trajectory without determining the diagnosis itself. More generally, the rule system constrains reasoning trajectories rather than encoding diagnostic decision trees, leaving the language model as the primary source of medical reasoning.

The symbols `|` and `||` denote alternative values inside a rule template. The template is expanded across these alternatives during compilation, producing multiple concrete rules. For example, `left || right` expands into two variants ("left" and "right"), and `right || left` expands into the corresponding opposite cases. Together these alternatives generate four concrete rules covering all left/right Weber lateralization combinations.

Figure 6 contrasts explicit decision-tree logic with local trajectory constraint during generation. This distinction is central: the rules operate as local constraints on generation rather than as a symbolic diagnostic program.

Appendix B: Representative rule targets in the study

The rulebook used in the experiments focuses on a small set of recurrent reasoning errors observed during baseline model generation. Rather than encoding a diagnostic decision tree, the rules target specific failure patterns that frequently appear in tuning-fork interpretation.

Table 9 summarizes the representative error patterns and the corresponding trajectory-editing intervention applied during decoding.

These interventions operate locally during generation. When a trigger pattern appears in the reasoning trajectory, the corresponding rule invalidates the unsafe continuation or inserts a protocol-compliant statement, allowing the language model to continue generating the explanation.

Appendix C: Reproducibility note and artifact map

Paper reproduction is exposed via `ae_paper_reproduction(...)` and a thin CLI wrapper (`python -m answer_engineering.repro.paper_ssnhl`). The run emits machine-readable artifacts including `results_summary.json`, `telemetry_summary.json`, `paper_tables.json`, and `run_manifest.json`. The manifest records commit hash, model/config, ruleset hash, validation-slice definition, and run configuration. Reported generation uses deterministic greedy decoding, so the primary reproducibility mechanism is the fixed model, dataset, ruleset, and decoding configuration rather than stochastic sampling.

Appendix D: Representative prompt snippets

To help clinicians verify that the evaluation cases reflect standard diagnostic reasoning, we include two representative prompt examples from the generated dataset. The dataset is produced from structured templates with controlled variations and injected secondary details intended to distract the model without changing the benchmark target.

Each clinical scenario has a paired contrast case with identical structure but inverted tuning-fork findings and class-consistent otoscopic examination, producing a different diagnosis and management decision.

Sudden sensorineural hearing loss (SSNHL) case.

Clinical prompt:

A 39-year-old patient presents with abrupt onset hearing loss in the left ear, noticed 60 hours ago. Hearing in that ear was unchanged before this change. The patient reports a history of seasonal allergies. Otoscopic examination shows unobstructed external

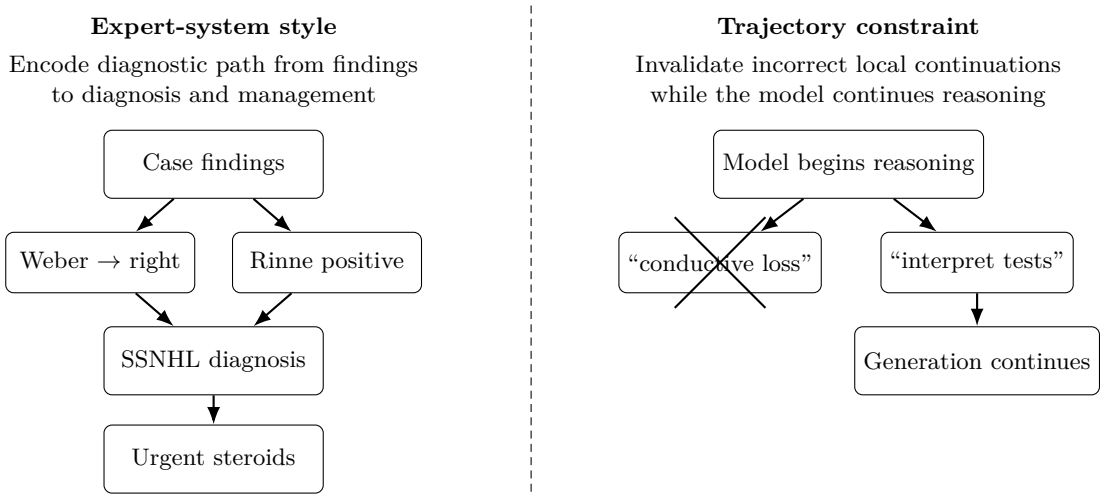

Figure 6: Comparison between explicit decision-tree logic and local trajectory constraint. Expert systems encode the diagnostic path directly, whereas Answer Engineering only removes invalid local continuations and leaves diagnosis generation to the language model.

Failure pattern	Typical incorrect continuation	Intervention
Contralateral Weber misinterpretation	Weber lateralizes to the opposite ear, followed by a conductive hearing loss interpretation	Invalidate the continuation and redirect reasoning to interpret Weber relative to the symptomatic ear.
Positive Rinne misinterpreted as conductive	Model states that air conduction exceeds bone conduction but still concludes conductive hearing loss	Invalidate the continuation and force reanalysis of the tuning-fork findings.
Premature diagnostic conclusion	Diagnosis generated before both ears have been evaluated	Redirect generation toward explicit analysis of Weber and Rinne findings in both ears.
Terminology inconsistency	Acute hearing loss described with nonstandard terminology	Normalize wording to the clinical term “sudden sensorineural hearing loss (SSNHL)”.
Protocol omission	SSNHL identified without mentioning urgency of treatment	Insert a statement emphasizing that treatment should be initiated urgently.

Table 9: Representative reasoning failures targeted by trajectory-editing rules in the experiments.

auditory canals and intact, normal-appearing tympanic membranes bilaterally. Tuning fork testing shows:

- Weber test lateralizes to the right ear
- Rinne test is positive on the left ear

The patient denies tinnitus, dizziness, vertigo, focal neurologic symptoms, or otalgia. Vital signs are normal. The patient appears calm during the examination. What is the best management at this time?

Correct reasoning:

- Weber lateralizes to the right ear (the ear opposite to the symptomatic side).
- Rinne is positive on the left (air conduction greater than bone conduction).
- Otoscopic examination is normal, making a conductive cause unlikely.
- The combined pattern is consistent with sudden sensorineural hearing loss (SSNHL).

Correct management: Prompt evaluation and a steroid recommendation consistent with the benchmark target for suspected SSNHL

Conductive contrast case.

Clinical prompt:
A 41-year-old patient presents with abrupt onset hearing loss in the left ear, noticed 46 hours ago. Hearing in that ear was unchanged before this onset. The patient reports a history of seasonal allergies. Otoscopic examination shows unobstructed external auditory canals and the left tympanic membrane appears subtly retracted but intact. Tuning fork testing demonstrates:

- Weber test lateralizes to the left ear
- Rinne test is negative on the left ear

The patient denies otalgia, dizziness, focal neurologic symptoms, tinnitus, or vertigo. Vital signs are within normal limits. What is the most appropriate management at this time?

Correct reasoning:

- Weber lateralizes to the left ear (the symptomatic ear).
- Rinne is negative on the left (bone conduction greater than air conduction).
- Otoscopic examination shows a retracted tympanic membrane, supporting a conductive process.
- The combined findings are consistent with conductive hearing loss in the affected ear.

Correct management: Management should address the likely conductive cause; the benchmark target does not call for steroid treatment for suspected SSNHL in this contrast case.

Injected distractor details Templates deliberately include secondary details that may distract the model from the protocol-critical findings used in this benchmark. Examples include:

- history of seasonal allergies
- mild nasal congestion
- normal vital signs
- calm appearance during examination

These elements are not treated as decisive for the benchmark label. The benchmark instead evaluates whether the model follows the consistent diagnostic pattern defined by tuning-fork results and otoscopic examination rather than over-weighting peripheral details.

Appendix E: Clinical justification of the benchmark endpoint

Guideline interpretation of Weber/Rinne findings. Bedside tuning-fork testing provides a rapid method to distinguish conductive from sensorineural hearing loss. The interpretation rules for Weber and Rinne tuning-fork testing are summarized in Table 3 of the AAO–HNS sudden hearing loss guideline issue (*Sudden Hearing Loss Guideline Issue* 2019). For the Weber test, "If the sound lateralizes to one ear, then: (a) There is CHL in that ear, or (b) There is SNHL in the opposite ear." For the Rinne test, the guideline states that "If the sound is heard better by bone conduction in the same ear, then there is CHL in that ear," whereas "If the sound is heard better by bone conduction but in the opposite ear, there is SNHL in the test ear." In standard clinical shorthand, a *positive Rinne* indicates air conduction greater than bone conduction, whereas a *negative Rinne* indicates bone conduction greater than air conduction. Accordingly, a presentation in which the Weber test *lateralizes to the contralateral ear* and the Rinne test remains *positive* in the symptomatic ear is consistent with *sensorineural hearing loss in the symptomatic ear*, whereas Weber lateralization to the symptomatic ear together with a *negative Rinne* indicates *conductive hearing loss*. The synthetic cases used in this study explicitly encode these guideline-described Weber and Rinne patterns to distinguish sensorineural from conductive hearing loss prior to audiometric confirmation.

Clinical justification of dataset assumptions. Otoscopic examination findings are specified in a class-consistent manner: they are normal in SSNHL-consistent cases and explicitly abnormal (e.g., retracted tympanic membrane) in conductive contrast cases. Additional non-decisive contextual details (e.g., allergies, nasal congestion, normal vital signs, calm appearance) are included as secondary distractors and do not determine the target decision boundary.

In this benchmark, the SSNHL setting defines a compact protocol boundary: the target management decision is determined by a consistent pattern of findings combining symptom timing, Weber/Rinne results, and aligned otoscopic examination. This is a benchmark design choice rather than a claim that real-world otologic management is fully determined by these cues alone.

In particular, normal otoscopic findings in SSNHL-consistent cases and abnormal findings in conductive cases are aligned with the corresponding tuning-fork patterns, ensuring internal consistency of the vignette

rather than introducing conflicting cues.

Benchmark design note. The dataset is intentionally template-generated rather than drawn from real-world case collections. The goal is not to model the full distribution of otologic practice, but to isolate a narrow protocol-sensitive distinction under controlled and internally consistent findings.

Templates are constructed as symmetric paired scenario families. SSNHL-consistent and conductive cases share similar surface structure, while decisive Weber/Rinne and otoscopic findings are changed in a class-consistent way. Symptom timing, Weber/Rinne patterns, and otoscopic findings are jointly specified so that the vignette supports a coherent branch (sensorineural vs. conductive) before management is evaluated. Otoscopic findings are aligned with class labels to avoid conflicting cues, while secondary details are included as distractors rather than label-defining variables.

Accordingly, this benchmark is intended for controlled comparative evaluation of protocol adherence under matched conditions, not for claims of broad clinical coverage, natural clinical prevalence, or physician-level diagnostic replacement.

Clinical rationale for endpoint selection. Evaluating clinical answers requires defining which elements of the management plan will serve as the benchmark signal. In suspected sudden sensorineural hearing loss (SSNHL), multiple management actions may appear in protocol-consistent responses, including urgent referral, audiometric confirmation, and steroid therapy.

Clinical practice guidelines describe SSNHL as a time-sensitive condition requiring urgent recognition and management. Guidelines recommend urgent specialist referral and prompt audiometric confirmation, and they recognize corticosteroids as first-line therapy once SSNHL is suspected (Chandrasekhar et al., 2019; National Institute for Health and Care Excellence, 2018; National Institute for Health and Care Excellence, 2019; Leung et al., 2016). Because SSNHL is time-sensitive, treatment should not normally be delayed solely to await confirmatory audiometry; referral, diagnostic testing, and treatment may proceed in parallel once the clinical suspicion is established from a consistent pattern of findings.

The inclusion of otoscopic findings is intended to establish a consistent clinical context and reduce ambiguity, not to replace specialist evaluation or downstream diagnostic confirmation.

Several benchmark signals could therefore be considered for evaluation. Table 10 summarizes candidate options and the rationale for selecting the final endpoint.

References

Arcuschin, I. et al. (2025). "Chain-of-Thought Reasoning in the Wild Is Not Always Faithful". In: *arXiv preprint arXiv:2503.08679*. Empirical study demonstrating that chain-of-thought explanations often diverge from the actual internal reasoning of large language models. URL: https://arxiv.org/abs/2503.08679.

Bai, Y. et al. (2022). "Constitutional AI: Harmlessness from AI Feedback". In: *arXiv preprint arXiv:2212.08073*. Introduces constitutional principles and self-critique for alignment via AI feedback. URL: https://arxiv.org/abs/2212.08073.

Beurer-Kellner, L., M. Fischer, and M. Vechev (2023). "LMQL: A Programming Language for Large Language Models". In: *Proceedings of the ACM on Programming Languages*. Introduces a query language for constrained and programmable language model generation. URL: https://arxiv.org/abs/2212.06094.

Chandrasekhar, S. S. et al. (Aug. 2019). "Clinical Practice Guideline: Sudden Hearing Loss (Update)". In: *Otolaryngology–Head and Neck Surgery* 161.1_suppl, S1–S45. DOI: 10.1177/0194599819859885. URL: https://doi.org/10.1177/0194599819859885.

Chen, J. et al. (Nov. 2025). "From Implicit Exploration to Structured Reasoning: Guideline and Refinement for LLMs". In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, pp. 3672–3684. DOI: 10.18653/v1/2025.findings-emnlp.196. URL: https://aclanthology.org/2025.findings-emnlp.196/.

Chen, Z. and C. Y. Suen (1994). "Measuring the Complexity of Rule-Based Expert Systems". In: *Expert Systems with Applications* 7.4, pp. 467–481. DOI: 10.1016/0957-4174(94)90072-8. URL: https://www.sciencedirect.com/science/article/pii/0957417494900728.

Deng, S. et al. (2023). "Towards a Unified View of Answer Calibration for Multi-Step Reasoning". In: *arXiv preprint arXiv:2311.09101*. Discusses calibration of answers in multi-step reasoning settings. URL: https://arxiv.org/abs/2311.09101.

Fast, D. et al. (2024). "Autonomous Medical Evaluation for Guideline Adherence of Large Language Models". In: *npj Digital Medicine* 7, p. 358. DOI: 10.1038/s41746-024-01356-6. URL: https://doi.org/10.1038/s41746-024-01356-6.

Gou, Z. et al. (2024). "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Cri-

Candidate signal	Limitation for benchmarking
Urgent referral	Appears in both protocol-consistent and protocol-inconsistent reasoning trajectories and therefore weakly discriminates protocol adherence.
Audiometry recommendation	Often present in protocol-consistent responses but may be omitted when treatment decisions are described first, limiting its usefulness as a benchmark signal.
Soft checklist (any management action: referral OR audiometry OR treatment)	Too permissive: many answers would satisfy the criterion without reaching the intended treatment decision, making it difficult to detect whether trajectory editing improves protocol adherence.
Composite checklist (referral AND audiometry AND treatment)	Too strict for this task: the question asks for the single best management step, while such a checklist would require a full management plan and penalize otherwise acceptable responses that omit additional actions not required by the benchmark.
Steroid recommendation	Directly reflects the treatment branch distinguishing suspected SSNHL from conductive hearing loss and therefore provides a clear protocol-critical signal.

Table 10: Candidate evaluation signals considered when designing the SSNHL benchmark.

tiquing". In: *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=Sx038qxjek.

Hager, P. et al. (2024). "Evaluation and Mitigation of the Limitations of Large Language Models in Clinical Decision-Making". In: *Nature Medicine* 30, pp. 2613–2622. DOI: 10.1038/s41591-024-03097-1. URL: https://doi.org/10.1038/s41591-024-03097-1.

Higa, K. (1996). "An Approach to Improving the Maintainability of Existing Rule Bases". In: *Decision Support Systems* 18.1, pp. 23–31. DOI: 10.1016/0167-9236(96)00015-2. URL: https://www.sciencedirect.com/science/article/pii/0167923696000152.

Hokamp, C. and Q. Liu (2017). "Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1535–1546. URL: https://aclanthology.org/P17-1141/.

Koo, T., F. Liu, and L. He (2024). "Automata-based Constraints for Language Model Decoding". In: *Proceedings of the Conference on Language Modeling (COLM)*. URL: https://openreview.net/forum?id=BDBdblmyzY.

Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the ACM Symposium on Operating Systems Principles*. Introduces vLLM, a high-throughput inference engine for large language models using paged attention memory management. URL: https://arxiv.org/abs/2309.06180.

Lanham, T. et al. (2023). "Measuring Faithfulness in Chain-of-Thought Reasoning". In: *arXiv preprint arXiv:2307.13702*. Introduces methods for evaluating whether reasoning traces produced by language models correspond to the model's internal reasoning process. URL: https://arxiv.org/abs/2307.13702.

Leung, M. A. et al. (2016). "Sudden Sensorineural Hearing Loss: Primary Care Update". In: *Canadian Family Physician* 62.9, pp. 711–713. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC4928516/.

Lightman, H. et al. (2023). *Let's Verify Step by Step*. arXiv: 2305.20050 [cs.LG]. URL: https://arxiv.org/abs/2305.20050.

Liu, Y. et al. (2023). "Trustworthy LLMs: A Survey and Guideline for Evaluating Large Language Models' Alignment". In: *arXiv preprint arXiv:2308.05374*. DOI: 10.48550/arXiv.2308.05374. URL: https://arxiv.org/abs/2308.05374.

Lyu, Q. et al. (2023). "Faithful Chain-of-Thought Reasoning". In: *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 305–329. DOI: 10.18653/v1/2023.ijcnlp-main.20. URL: https://aclanthology.org/2023.ijcnlp-main.20/.

Madaan, A. et al. (Mar. 2023). "Self-Refine: Iterative Refinement with Self-Feedback". In: *arXiv preprint*

arXiv:2303.17651. arXiv: 2303.17651 [cs.CL]. URL: https://arxiv.org/abs/2303.17651.

Miao, N. et al. (2024). "SelfCheck: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning". In: *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=pTHfApDakA.

National Institute for Health and Care Excellence (2018). *Hearing Loss in Adults: Assessment and Management*. NICE Guideline NG98. URL: https://www.nice.org.uk/guidance/ng98.

– (2019). *Hearing Loss in Adults: Quality Standard 2 – Sudden Onset of Hearing Loss*. NICE Quality Standard QS185. URL: https://www.nice.org.uk/guidance/qs185/chapter/quality-statement-2-sudden-onset-of-hearing-loss.

NVIDIA (2023). "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications". In: *arXiv preprint arXiv:2308.12372*. Describes a framework for defining safety, policy, and workflow guardrails for large language model applications. URL: https://arxiv.org/abs/2308.12372.

OpenMeditron (2025). *OpenMeditron/Meditron3-8B*. https://huggingface.co/OpenMeditron/Meditron3-8B. Hugging Face model card, accessed 2026-03-15.

Paul, D. et al. (2024). "Making Reasoning Matter: Measuring and Improving Faithfulness of Chain-of-Thought Reasoning". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. URL: https://aclanthology.org/2024.findings-emnlp.882/.

Reddy, B. and R. Fields (2022). "From Past to Present: A Comprehensive Technical Review of Rule-Based Expert Systems from 1980–2021". In: *Proceedings of the 2022 ACM Southeast Conference*. DOI: 10.1145/3476883.3520211. URL: https://dl.acm.org/doi/10.1145/3476883.3520211.

Research, M. (2023). "Guidance: A Language for Controlling Large Language Models". In: *arXiv preprint arXiv:2307.11760*. Introduces a programmable interface for constrained decoding and structured language model control. URL: https://arxiv.org/abs/2307.11760.

Roeschl, T. et al. (Nov. 2025). "Assessing the Limitations of Large Language Models in Clinical Practice Guideline-Concordant Treatment Decision-Making on Real-World Data: Retrospective Study". In: *JMIRx Med* 6, e74899. DOI: 10.2196/74899. URL: https://doi.org/10.2196/74899.

Shi, W. et al. (2023). "Trusting Your Evidence: Hallucinate Less with Context-Aware Decoding". In: *arXiv preprint arXiv:2305.14739*. Decoding strategies that incorporate evidence-aware control to reduce hallucinations. URL: https://arxiv.org/abs/2305.14739.

Shinn, N. et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Introduces an iterative agent framework that uses verbal feedback and reflection across attempts. URL: https://openreview.net/forum?id=vAElhFcKW6.

Srivatsa, V. et al. (2024). "DeAL: Decoding-Time Alignment for Large Language Models". In: *arXiv preprint arXiv:2402.06147*. Introduces decoding-time interventions to align model outputs with constraints. URL: https://arxiv.org/abs/2402.06147.

Sudden Hearing Loss Guideline Issue (2019). AAO–HNS clinical practice guideline summary. Guideline Central. URL: https://eguideline.guidelinecentral.com/i/327549-sudden-hearing-loss/7.

Turpin, M. et al. (2023). "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting". In: *arXiv preprint arXiv:2305.04388*. URL: https://arxiv.org/abs/2305.04388.

Tyen, G. et al. (2024). "LLMs Cannot Find Reasoning Errors, but Can Correct Them Given the Error Location". In: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, pp. 13894–13908. DOI: 10.18653/v1/2024.findings-acl.826. URL: https://aclanthology.org/2024.findings-acl.826/.

Vijayakumar, A. K. et al. (2018). "Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. URL: https://arxiv.org/abs/1610.02424.

Wang, X. et al. (2023). "Self-Consistency Improves Chain of Thought Reasoning in Language Models". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2203.11171.

Wei, J. et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2201.11903.

Wiegreffe, S., S. Tiwary, N. A. Smith, et al. (2024). "Faithfulness vs. Plausibility: On the (Un)Reliability of Explanations from Large Language Models". In: *arXiv preprint arXiv:2402.04614*. URL: https://arxiv.org/abs/2402.04614.

Yang, K. and D. Klein (2021). "FUDGE: Controlled Text Generation with Future Discriminators". In: *Proceedings of NAACL-HLT*. Introduces discriminator-guided decoding for controllable text generation. Association for Computational Linguistics. URL: https://arxiv.org/abs/2104.05218.

Yao, S., D. Yu, et al. (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *arXiv preprint arXiv:2305.10601*. Introduces search-based reasoning over multiple reasoning trajectories. URL: https://arxiv.org/abs/2305.10601.

Yao, S., J. Zhao, et al. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. Combines reasoning traces with tool usage for interactive problem solving. URL: https://arxiv.org/abs/2210.03629.

Yun, J. et al. (2025). “Med-PRM: Medical Reasoning Models with Stepwise, Guideline-verified Process Rewards”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. URL: https://aclanthology.org/2025.emnlp-main.837/.

Zelikman, E. et al. (2022). “STaR: Self-Taught Reasoner Bootstrapping Reasoning With Reasoning”. In: *arXiv preprint*. Introduces iterative self-improvement of reasoning trajectories via self-generated rationales. arXiv: 2203.14465 [cs.CL]. URL: https://arxiv.org/abs/2203.14465 (visited on 03/27/2026).

Zhang, Y. et al. (2023). “A Survey of Constrained Text Generation for Large Language Models”. In: *arXiv preprint arXiv:2309.15071*. URL: https://arxiv.org/abs/2309.15071.

Zhao, H. et al. (2024). “Controllable Text Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2408.12599*. URL: https://arxiv.org/abs/2408.12599.

Zheng, L. et al. (2024). “SGLang: Efficient Execution of Structured Language Model Programs”. In: *arXiv preprint arXiv:2312.07104*. Presents a runtime system for structured language model programs with optimized scheduling and constrained generation. URL: https://arxiv.org/abs/2312.07104.